

CET138 Assignment 1 — Concepts Report

A walk-through of the Full Stack, HTML, CSS, Bootstrap and JavaScript concepts used in the **Eventide** Event Booking website, with code pulled straight from the project files.

1. Full Stack Development

Full stack development just means a developer can work on both halves of a website. The **front-end** is everything you see and click — built with HTML, CSS and JavaScript and running in your browser. The **back-end** is the server-side bit that handles logins, processes data and talks to a **database**, which is where information is actually stored long term. The three pieces talk to each other over an API: the browser sends a request, the server runs some logic and asks the database for what it needs, then sends an answer back.

What I built

This project is front-end only — there is no real server. The **booking form** is the part that would talk to a back-end in a production version: a Node/Express API would receive the booking, validate it, store the record in a database like MongoDB or MySQL, and email a confirmation. For the assignment, JavaScript fakes that last step by showing a confirmation modal.

Code from the project

```
form.addEventListener('submit', (e) => {
  e.preventDefault();

  const name = document.getElementById('full-name').value.trim();
  const email = document.getElementById('email').value.trim();
  const eventName = eventSelect.value;
  const qty = parseInt(quantityInput.value, 10) || 0;
  const price = parseFloat(eventSelect.options[eventSelect.selectedIndex].dataset.price) || 0;
  const total = (price * qty).toFixed(2);

  if (!name || !email || !eventName || qty < 1) {
    alert('Please fill in all the fields before confirming.');
```

```
    return;
  }

  modalBody.innerHTML = `
    <p>Thanks <strong>${escapeHtml(name)}</strong>. A confirmation will be sent to <strong>${escapeHtml(email)}</strong>.</p>
    <div class="summary-row"><span>Event</span><strong>${escapeHtml(eventName)}</strong></div>
    <div class="summary-row"><span>Tickets</span><strong>${qty}</strong></div>
    <div class="summary-row"><span>Total paid</span><strong>Rs. ${total}</strong></div>
  `;

  confirmModal.show();
}
```

What the code does

preventDefault() stops the browser from doing its usual full-page reload on submit, so the JavaScript can take over. The next few lines collect the values the user typed in — name, email, event, quantity — exactly the kind of payload a real API would receive. The **if** block does basic

client-side validation so we don't try to confirm an empty booking. In a real full stack version, the next step would be a `fetch()` call sending this data to a server endpoint; here, the modal stands in for that response.

Why I did it this way

Splitting validation between the browser and the server is standard practice — the front-end catches obvious mistakes quickly, and the back-end does the proper, trusted check before anything ever reaches a database.

2. HTML

HTML is the layer that gives a webpage its structure. It is not a programming language — there are no loops or calculations — it just labels each chunk of content so the browser knows what it is. Modern HTML5 also gives us **semantic tags** like `<header>`, `<nav>`, `<main>` and `<footer>`, which describe the role of each section. This matters for accessibility (screen readers can navigate the page properly) and for SEO (Google understands the page layout better).

What I built

The whole Eventide page uses semantic tags rather than a wall of `<div>`s. The top of the page is wrapped in `<header>`, the main listing in `<main>`, each event in its own `<article>`, and the bottom in `<footer>`.

Code from the project

```
<header>
  <nav class="navbar navbar-expand-lg navbar-dark fixed-top site-nav">
    <div class="container">
      <a class="navbar-brand fw-bold" href="#home">Eventide</a>
      <button class="navbar-toggler" type="button" data-bs-toggle="collapse"
        data-bs-target="#mainNav" aria-controls="mainNav"
        aria-expanded="false" aria-label="Toggle navigation">
        <span class="navbar-toggler-icon"></span>
      </button>
      <div class="collapse navbar-collapse" id="mainNav">
        <ul class="navbar-nav ms-auto">
          <li class="nav-item"><a class="nav-link" href="#home">Home</a></li>
          <li class="nav-item"><a class="nav-link" href="#events">Events</a></li>
          <li class="nav-item"><a class="nav-link" href="#book">Book Now</a></li>
          <li class="nav-item"><a class="nav-link" href="#contact">Contact</a></li>
        </ul>
      </div>
    </div>
  </nav>

  <section id="home" class="hero">
    <div class="container hero-inner">
      <h1 class="hero-title">Book live events around Nepal.</h1>
      <p class="hero-sub">Music, food, tech and more. Pick what you like and book your ticket online.</p>
      <a href="#events" class="btn btn-primary btn-lg hero-cta">See Events</a>
    </div>
  </section>
</header>
```

What the code does

The outer `<header>` groups everything at the top of the page — the site-wide `<nav>` bar and the hero `<section>` with the welcome message and call-to-action. Using a real `<nav>` rather than a generic `<div>` tells screen readers "this is the site navigation", and the four `<a>` tags inside the unordered list jump to the matching sections of the page using their id attributes as anchors.

Why I did it this way

Semantic HTML costs nothing extra to write and makes the site more accessible and easier for search engines to index — it is the modern default for good reason.

3. CSS

CSS — Cascading Style Sheets — is what makes a page actually look like something. HTML lays the bones down; CSS picks the colours, fonts, spacing and layout. **CSS variables** (properties defined with `--name` inside `:root`) let you define a value once and reuse it everywhere, so updating the whole theme is a one-line change.

What I built

The whole Eventide colour palette and spacing scale lives in a single `:root` block at the top of `style.css`. The rest of the stylesheet pulls those values in with `var(--name)`, so anything from the hero banner to the booking form picks up the same accent red and rounded corners.

Code from the project

```
:root {
  --bg: #f7f7f5;
  --surface: #ffffff;
  --surface-alt: #f0eee9;
  --text-primary: #111111;
  --text-secondary: #555555;
  --accent: #d33c15;
  --accent-dark: #a62e0e;
  --border: #e2e0db;
  --radius: 14px;
  --space-sm: 0.75rem;
  --space-md: 1.5rem;
  --space-lg: 3rem;
  --space-xl: 5rem;
  --shadow: 0 8px 24px rgba(17, 17, 17, 0.06);
}
```

What the code does

`:root` is the top of the CSS tree — variables defined here are available to every element on the page. `--bg` and `--surface` are the two main background tones, `--accent` is the brand red used on buttons and hover states, and the `--space-*` values give a consistent rhythm so padding looks the same across sections. The `--radius` and `--shadow` variables keep card corners and drop shadows uniform.

Why I did it this way

Without variables, swapping the accent colour later would mean hunting through the whole stylesheet — with them, it's a one-line change that the rest of the site picks up automatically.

4. Bootstrap

Bootstrap is a CSS framework that gives you ready-made, responsive components — navbars, grids, forms, modals — out of the box. Instead of writing media queries and layout maths yourself, you drop classes like `col-md-6` onto your HTML and Bootstrap handles the rest, including how things rearrange on phones.

What I built

Bootstrap powers the Eventide **navbar with mobile hamburger collapse**, the **responsive 3-column event grid** that stacks on small screens, the **booking form layout** (`col-md-6` fields side by side), and the **confirmation modal**.

Code from the project

```
<nav class="navbar navbar-expand-lg navbar-dark fixed-top site-nav">
  <div class="container">
    <a class="navbar-brand fw-bold" href="#home">Eventide</a>
    <button class="navbar-toggler" type="button" data-bs-toggle="collapse"
      data-bs-target="#mainNav" aria-controls="mainNav"
      aria-expanded="false" aria-label="Toggle navigation">
      <span class="navbar-toggler-icon"></span>
    </button>
    <div class="collapse navbar-collapse" id="mainNav">
      <ul class="navbar-nav ms-auto">
        <li class="nav-item"><a class="nav-link" href="#home">Home</a></li>
        <li class="nav-item"><a class="nav-link" href="#events">Events</a></li>
        <li class="nav-item"><a class="nav-link" href="#book">Book Now</a></li>
        <li class="nav-item"><a class="nav-link" href="#contact">Contact</a></li>
      </ul>
    </div>
  </div>
</nav>
```

What the code does

`navbar-expand-lg` tells Bootstrap to keep the menu expanded on large screens and collapse it on smaller ones. The `navbar-toggler` button is the hamburger that appears on mobile — its `data-bs-toggle="collapse"` and `data-bs-target="#mainNav"` attributes wire it up to the menu without any JavaScript I have to write. Inside the collapsible div, the four nav links use `nav-link` classes so they pick up Bootstrap's default styling on top of my own overrides in CSS.

Why I did it this way

Writing a fully accessible, responsive navbar from scratch is a lot of work — Bootstrap solves it in a handful of classes, which lets me focus on the bits of the site that are actually specific to Eventide.

5. JavaScript

JavaScript is what makes a webpage actually *do* things. HTML and CSS only describe what's on screen; JavaScript reacts to clicks, key presses and form submissions, updates the page without reloading, and can talk to a back-end over the network. It works by listening for **events** and then changing the **DOM** — the live tree of elements the browser is showing.

What I built

Two of the most interactive bits of Eventide are pure JavaScript: the **live search filter** that hides and shows event cards as you type, and the **live total price** in the booking form that updates the moment you change the quantity or the event. The booking modal is also built dynamically — its summary HTML is generated by JavaScript at the point of submit.

Code from the project

```
searchInput.addEventListener('input', (e) => {
  const term = e.target.value.trim().toLowerCase();
  let visibleCount = 0;

  eventCards.forEach(card => {
    const name = card.dataset.name.toLowerCase();
    const matches = name.includes(term);
    card.parentElement.style.display = matches ? '' : 'none';
    if (matches) visibleCount++;
  });

  noResults.classList.toggle('d-none', visibleCount > 0);
});
```

What the code does

`addEventListener('input', ...)` fires on every keystroke, so the filter feels instant. Inside, I lowercase the search term once, then loop through every event card and check whether its `data-name` attribute contains the term. Cards that match stay visible; non-matches have their column parent's `display` set to `none` so the grid re-flows neatly. A small counter tracks how many cards are still visible, and a Bootstrap `d-none` class is toggled on a "no results" message accordingly.

Why I did it this way

Filtering on the front-end is fine here because the dataset is tiny — six cards. For thousands of events I'd send the search term to a server and let the database do the filtering instead, which is much more efficient at scale.